

# Netflix — 30 DSA Interview Practice Questions

Targeted practice set for software-engineering interview preparation. Problems are LeetCode-style and can be practiced on LeetCode, GeeksforGeeks, HackerRank, CodeSignal, CodeChef or equivalent platforms. This is a preparation list based on public interview patterns; it is not a claim that these exact questions will be asked.

Company-specific questions vary by role, level, location and interview cycle. Use this as a targeted preparation set, not an exact-question guarantee.

| #  | Problem                                        | Topic                  | Difficulty | Platform       |
|----|------------------------------------------------|------------------------|------------|----------------|
| 1  | Climbing Stairs                                | DP                     | Easy       | LeetCode-style |
| 2  | Accounts Merge                                 | Graph/DSU              | Medium     | LeetCode-style |
| 3  | Binary Tree Level Order Traversal              | Tree/BFS               | Medium     | LeetCode-style |
| 4  | Cheapest Flights Within K Stops                | Graph/DP               | Medium     | LeetCode-style |
| 5  | Clone Graph                                    | Graph                  | Medium     | LeetCode-style |
| 6  | Coin Change                                    | DP                     | Medium     | LeetCode-style |
| 7  | Course Schedule                                | Graph/Topological Sort | Medium     | LeetCode-style |
| 8  | Course Schedule II                             | Graph/Topological Sort | Medium     | LeetCode-style |
| 9  | Decode Ways                                    | DP                     | Medium     | LeetCode-style |
| 10 | House Robber                                   | DP                     | Medium     | LeetCode-style |
| 11 | Kth Largest Element in an Array                | Heap/Quickselect       | Medium     | LeetCode-style |
| 12 | LRU Cache                                      | Design/HashMap         | Medium     | LeetCode-style |
| 13 | Longest Common Subsequence                     | DP                     | Medium     | LeetCode-style |
| 14 | Longest Increasing Subsequence                 | DP                     | Medium     | LeetCode-style |
| 15 | Longest Substring Without Repeating Characters | Sliding Window         | Medium     | LeetCode-style |
| 16 | Lowest Common Ancestor of a Binary Tree        | Tree                   | Medium     | LeetCode-style |
| 17 | Meeting Rooms II                               | Intervals/Heap         | Medium     | LeetCode-style |
| 18 | Network Delay Time                             | Graph/Dijkstra         | Medium     | LeetCode-style |
| 19 | Number of Islands                              | Graph/BFS-DFS          | Medium     | LeetCode-style |
| 20 | Pacific Atlantic Water Flow                    | Graph/DFS              | Medium     | LeetCode-style |
| 21 | Rotting Oranges                                | Graph/BFS              | Medium     | LeetCode-style |
| 22 | Top K Frequent Elements                        | Heap/Hashing           | Medium     | LeetCode-style |
| 23 | Ugly Number II                                 | DP/Heap                | Medium     | LeetCode-style |
| 24 | Word Break                                     | DP                     | Medium     | LeetCode-style |
| 25 | Binary Tree Maximum Path Sum                   | Tree/DFS               | Hard       | LeetCode-style |
| 26 | Edit Distance                                  | DP                     | Hard       | LeetCode-style |
| 27 | Merge K Sorted Lists                           | Heap/Linked List       | Hard       | LeetCode-style |
| 28 | Minimum Window Substring                       | Sliding Window         | Hard       | LeetCode-style |
| 29 | Serialize and Deserialize Binary Tree          | Tree/Design            | Hard       | LeetCode-style |
| 30 | Word Ladder                                    | Graph/BFS              | Hard       | LeetCode-style |

**Interview practice rule:** For each problem, first explain brute force, then optimize it. Always state time complexity, space complexity, assumptions and edge cases. After solving, practice one variation of the same pattern.